

Lab 9: Transaction Control Language (TCL) and Data Control Language (DCL)

Name: Shayan Madrekar

Roll Number: E041

Date: 10/04/2026

Question 1a:

Query: BEGIN;

INSERT INTO employee VALUES

('E010', 'Test Emp1', 'D01', 60000, '2024-01-01'),

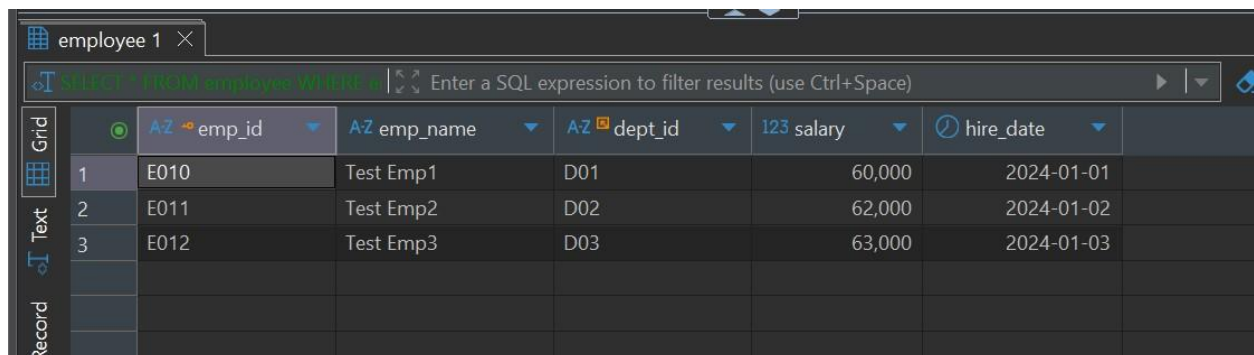
('E011', 'Test Emp2', 'D02', 62000, '2024-01-02'),

('E012', 'Test Emp3', 'D03', 63000, '2024-01-03');

SELECT * FROM employee WHERE emp_id IN ('E010','E011','E012');

ROLLBACK;

Output: Before rollback



	emp_id	emp_name	dept_id	salary	hire_date
1	E010	Test Emp1	D01	60,000	2024-01-01
2	E011	Test Emp2	D02	62,000	2024-01-02
3	E012	Test Emp3	D03	63,000	2024-01-03

After Rollback

```
SELECT * FROM employee WHERE emp_id IN ('E010','E011','E012');
```

```
ROLLBACK;
```

Employee 1 X

FROM employee WHERE Enter a SQL expression to filter results (use Ctrl+Space)

A-Z emp_id	A-Z emp_name	A-Z dept_id	123 salary	hire_date

Question 1b:

Query:

```
BEGIN;
```

```
INSERT INTO department VALUES ('D05', 'Sales', 'Delhi');
```

```
SAVEPOINT sp1;
```

```
INSERT INTO department VALUES ('D06', 'Support', 'Chennai');
```

```
ROLLBACK TO sp1;
```

```
COMMIT;
```

```
SELECT * FROM department WHERE dept_id IN ('D05','D06');
```

Output:

department 1 X

SELECT * FROM department WHERE Enter a SQL expression to filter results (use Ctrl+Space)

	A-Z dept_id	A-Z dept_name	A-Z location
1	D05	Sales	Delhi

Explanation: Only 1 department was added (D05).

Question 1c:

Query:

```

BEGIN;

UPDATE employee SET salary = salary * 1.10 WHERE dept_id = 'D01';

SAVEPOINT sp1;

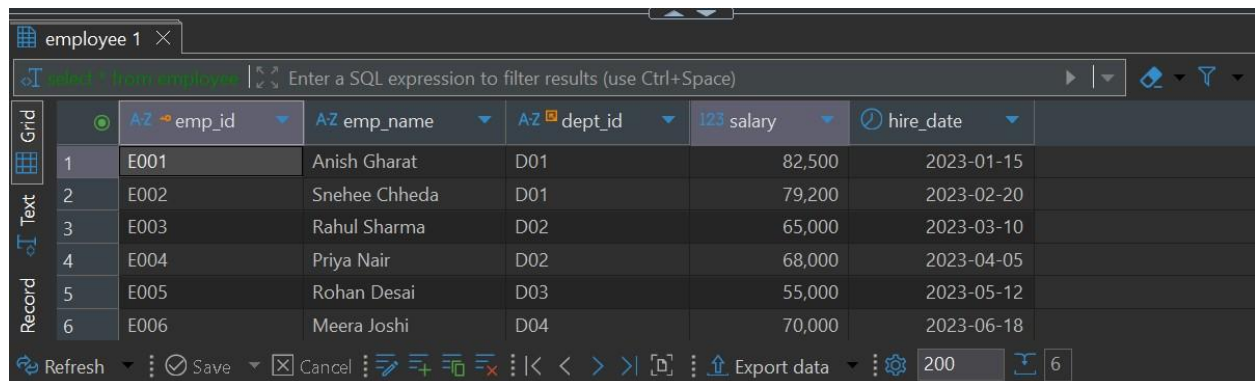
UPDATE employee SET salary = salary * 1.05 WHERE dept_id = 'D02';

ROLLBACK TO sp1;

COMMIT;

```

Output:



	emp_id	emp_name	dept_id	salary	hire_date
1	E001	Anish Gharat	D01	82,500	2023-01-15
2	E002	Snehee Chheda	D01	79,200	2023-02-20
3	E003	Rahul Sharma	D02	65,000	2023-03-10
4	E004	Priya Nair	D02	68,000	2023-04-05
5	E005	Rohan Desai	D03	55,000	2023-05-12
6	E006	Meera Joshi	D04	70,000	2023-06-18

Explanation: Only department D01 had permanent salary changes. The salary of employees in department D01 is increased first and then a savepoint is created. After that, salaries in D02 are updated. When we rollback to the savepoint, all changes made after it (i.e., D02 update) are undone. The commit ensures that only the D01 changes are saved permanently.

Question 2a:

Query:

```

BEGIN;

INSERT INTO project VALUES ('P004', 'New Project', 400000, '2024-05-01');
SAVEPOINT sp_project;

INSERT INTO emp_project VALUES ('E001', 'P004', 50);

SAVEPOINT sp_assignment;

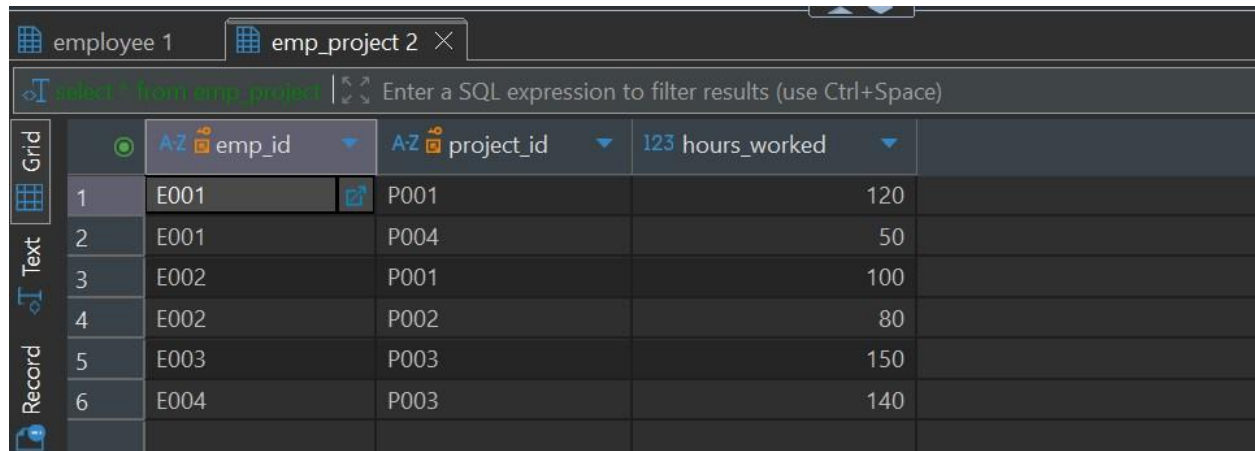
UPDATE project SET budget = 450000 WHERE project_id = 'P004';

```

```
ROLLBACK TO sp_assignment;
```

```
COMMIT;
```

Output:



	emp_id	project_id	hours_worked
1	E001	P001	120
2	E001	P004	50
3	E002	P001	100
4	E002	P002	80
5	E003	P003	150
6	E004	P003	140

Explanation:

Saved Changes:

- Project P004 inserted
- Employee assignment saved
- Budget update NOT saved

The project is inserted and saved before any rollback occurs. Then, an employee is assigned to the project, followed by another savepoint. The budget update happens after the second savepoint. When we rollback to sp_assignment, only the budget update is undone. Since the transaction is committed afterward, the project and assignment remain, but the updated budget does not.

Question 2b:

Query:

```
BEGIN;
```

```
INSERT INTO department VALUES ('D07', 'Legal', 'Mumbai');  
SAVEPOINT sp1;
```

```

INSERT INTO department VALUES ('D08', 'Admin', 'Pune');
SAVEPOINT sp2;

INSERT INTO department VALUES ('D09', 'Ops', 'Delhi');
SAVEPOINT sp3;

ROLLBACK TO sp2; COMMIT;

```

Output:

	dept_id	dept_name	location
1	D01	Engineering	Mumbai
2	D02	Marketing	Pune
3	D03	HR	Mumbai
4	D04	Finance	Bangalore
5	D05	Sales	Delhi
6	D07	Legal	Mumbai
7	D08	Admin	Pune

Explanation: When rolling back to the middle savepoint (sp2), all changes made after that savepoint are undone. This includes the insert related to sp3. Additionally, the savepoint sp3 itself is also removed. Only the operations up to sp2 remain valid, meaning D07 and D08 persist, while D09 is discarded.

Question 3a:

Query:

```

BEGIN;

DELETE FROM employee WHERE dept_id = 'D03';

SELECT * FROM employee WHERE dept_id = 'D03';

ROLLBACK;

SELECT * FROM employee WHERE dept_id = 'D03';

```

Output:

After Delete

	emp_id	emp_name	dept_id	salary	hire_date

After Rollback

	emp_id	emp_name	dept_id	salary	hire_date
1	E005	Rohan Desai	D03	55,000	2023-05-12

Explanation: After deleting employees from department D03, querying before commit shows that no records exist for that department, as the deletion is visible within the transaction. However, since the transaction is not committed, performing a rollback restores the deleted records. After rollback, querying again shows the original employees, proving that uncommitted changes are reversible.

Question 3b:

Query:

```
BEGIN;

UPDATE employee SET salary = 80000 WHERE emp_id = 'E001';

CREATE TABLE test_table (
id INT
);

COMMIT;
```

Output:

employee 1

employee 2

select * from employee

Enter a SQL expression to filter results (use Ctrl+Space)

Grid

Text

Record

	AZ emp_id	AZ emp_name	AZ dept_id	123 salary	hire_date
1	E001	Anish Gharat	D01	80,000	2023-01-15
2	E002	Snehee Chheda	D01	79,200	2023-02-20
3	E003	Rahul Sharma	D02	65,000	2023-03-10
4	E004	Priya Nair	D02	68,000	2023-04-05
5	E005	Rohan Desai	D03	55,000	2023-05-12
6	E006	Meera Joshi	D04	70,000	2023-06-18

Refresh

Save

Cancel

Export data

200

6

Explanation: In most database systems like MySQL, executing a DDL statement such as CREATE TABLE causes an implicit commit. This means that the salary update is automatically committed before the table creation. As a result, even if we intended to control the transaction manually, the update becomes permanent due to the implicit commit behavior of DDL operations.

Question 4a:

Query:

```
CREATE USER analyst IDENTIFIED BY 'pass123';
CREATE USER data_entry IDENTIFIED BY 'pass123';

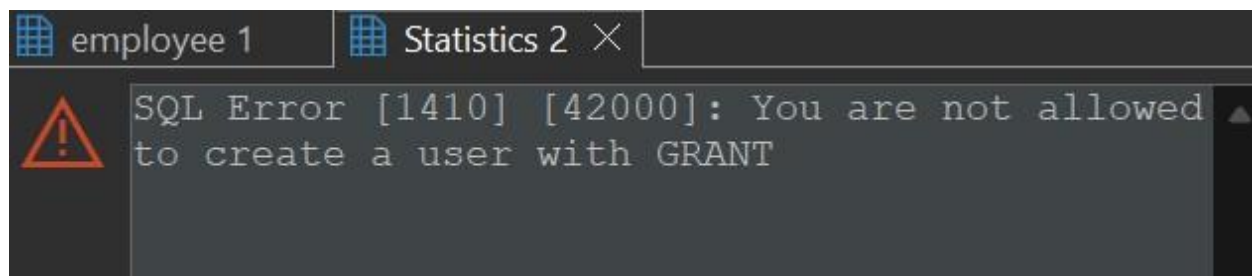
GRANT SELECT ON employee TO analyst;

GRANT INSERT, UPDATE ON employee TO data_entry;
```

Question 4b:

Query: `GRANT DELETE ON employee TO unknown user;`

Output:



Explanation: The database throws an error because privileges can only be granted to existing users. Since the specified user does not exist, the GRANT command fails.

Question 4c:

Query:

```
GRANT ALL ON project TO analyst;  
  
REVOKE DELETE ON project FROM analyst;  
  
SHOW GRANTS FOR analyst;
```

Output:

The screenshot shows the SQL editor with the following content:

```

employee 1 Results 2 X
--
Enter a SQL expression to filter results (use Ctrl+Space)
--
A-Z Grants for analyst@%
--
1 G Connection: localhost
2 G Time: 2026-04-10 19:41:45.377 O `analyst`@`%`
3 GRANT SELECT, INSERT, UPDATE, CREATE, DROP, REFERENCES, INDEX, ALTER, CREATE VIEW, SHOW VIEW, TRIGGER ON `emp`.`project` TO `analyst`@`%`

```

Explanation: analyst has all privileges except DELETE. Initially, all privileges are granted to the analyst user. When the DELETE privilege is revoked, the user retains all other privileges such as SELECT, INSERT, and UPDATE. The SHOW GRANTS command confirms the remaining permissions.

Question 5a:

Query:

```
CREATE USER temp_user IDENTIFIED BY 'pass123';

GRANT SELECT, INSERT ON employee TO temp_user;

BEGIN;

INSERT INTO employee VALUES ('E013', 'Temp Emp', 'D01', 60000, '2024-01-01');
REVOKE INSERT ON employee FROM temp_user;

ROLLBACK;
```

Output:

	emp_id	emp_name	dept_id	salary	hire_date
1	E001	Anish Gharat	D01	80,000	2023-01-15
2	E002	Snehee Chheda	D01	79,200	2023-02-20
3	E003	Rahul Sharma	D02	65,000	2023-03-10
4	E004	Priya Nair	D02	68,000	2023-04-05
5	E005	Rohan Desai	D03	55,000	2023-05-12
6	E006	Meera Joshi	D04	70,000	2023-06-18
7	E013	Temp Emp	D01	60,000	2024-01-01

Explanation: temp_user CANNOT insert anymore. Even though the transaction is rolled back, the REVOKE operation still takes effect. This is because DCL commands are auto-committed and are not affected by transaction control commands like rollback. Therefore, temp_user permanently loses the INSERT privilege.

Question 5b:

Explanation: DCL operations such as GRANT and REVOKE are not affected by ROLLBACK because they are auto-committed by the database system. These operations modify access control and security settings, which are considered critical and must take immediate effect. Therefore, they are not treated as transactional operations like DML commands (INSERT, UPDATE, DELETE).

Question 6a:

Query:

```
BEGIN;

INSERT INTO employee VALUES ('E020','Invalid Emp','D99',50000,'2024-01-01');
SAVEPOINT sp1;

INSERT INTO employee VALUES ('E021','Valid Emp','D01',55000,'2024-01-02');
ROLLBACK TO sp1;

COMMIT;
```

Output:

	emp_id	emp_name	dept_id	salary	hire_date
2	E002	Snehee Chheda	D01	79,200	2023-02-20
3	E003	Rahul Sharma	D02	65,000	2023-03-10
4	E004	Priya Nair	D02	68,000	2023-04-05
5	E005	Rohan Desai	D03	55,000	2023-05-12
6	E006	Meera Joshi	D04	70,000	2023-06-18
7	E013	Temp Emp	D01	60,000	2024-01-01

Explanation: The first insert fails because department D99 does not exist, violating the foreign key constraint. The savepoint ensures that subsequent operations can be controlled. When rolling back to the savepoint, any changes after it are undone. This demonstrates how savepoints help in partial recovery from errors without discarding the entire transaction.

Question 6b:

Query:

```
BEGIN;

UPDATE employee SET dept_id = 'D99' WHERE emp_id = 'E001';

ROLLBACK;
```

Output:

	emp_id	emp_name	dept_id	salary	hire_date
1	E001	Anish Gharat	D01	80,000	2023-01-15
2	E002	Snehee Chheda	D01	79,200	2023-02-20
3	E003	Rahul Sharma	D02	65,000	2023-03-10
4	E004	Priya Nair	D02	68,000	2023-04-05
5	E005	Rohan Desai	D03	55,000	2023-05-12
6	E006	Meera Joshi	D04	70,000	2023-06-18
7	E013	Temp Emp	D01	60,000	2024-01-01

Explanation: Updating the department ID to a non-existent value violates the foreign key constraint, causing an error. Using transaction control ensures that the invalid update does not affect the database permanently. The rollback restores the original valid state.

Question 7a:

Query:

```

BEGIN;

INSERT INTO employee VALUES ('E020','Test Employee','D01',50000,'2024-08-01');
SAVEPOINT sp1;

-- Avoid DROP TABLE here

ROLLBACK TO sp1;

COMMIT;

```

Output:

	emp_id	emp_name	dept_id	salary	hire_date
2	E002	Snehee Chheda	D01	79,200	2023-02-20
3	E003	Rahul Sharma	D02	65,000	2023-03-10
4	E004	Priya Nair	D02	68,000	2023-04-05
5	E005	Rohan Desai	D03	55,000	2023-05-12
6	E006	Meera Joshi	D04	70,000	2023-06-18
7	E013	Temp Emp	D01	60,000	2024-01-01
8	E020	Test Employee	D01	50,000	2024-08-01

Explanation: DROP TABLE removes table permanently and cannot be rolled back. DDL operations like DROP TABLE often cause implicit commits and cannot be reversed using rollback. This leads to unexpected behavior. The correct approach is to avoid such operations within transactional logic.

Question 7b:

Query:

```

BEGIN;

UPDATE employee SET salary = salary * 1.1 WHERE dept_id = 'D01';

SAVEPOINT sp1;

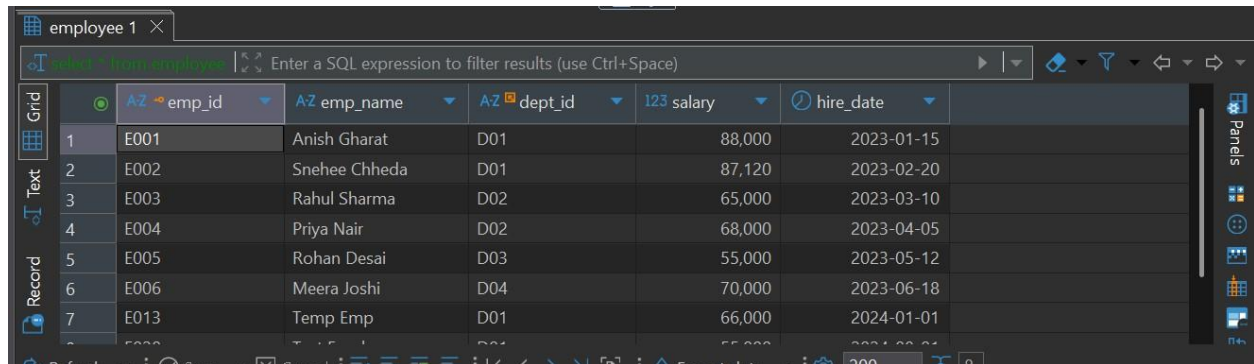
UPDATE employee SET salary = salary * 1.05 WHERE dept_id = 'D02';

ROLLBACK TO sp1;

```

```
COMMIT;
```

Output:



The screenshot shows a database query result in a dark-themed interface. The query bar at the top contains 'select * from employee'. Below the query bar, a table of employee data is displayed. The table has columns for emp_id, emp_name, dept_id, salary, and hire_date. The data is as follows:

	emp_id	emp_name	dept_id	salary	hire_date
1	E001	Anish Gharat	D01	88,000	2023-01-15
2	E002	Snehee Chheda	D01	87,120	2023-02-20
3	E003	Rahul Sharma	D02	65,000	2023-03-10
4	E004	Priya Nair	D02	68,000	2023-04-05
5	E005	Rohan Desai	D03	55,000	2023-05-12
6	E006	Meera Joshi	D04	70,000	2023-06-18
7	E013	Temp Emp	D01	66,000	2024-01-01

Explanation: SAVEPOINT cannot be used after COMMIT. Once a COMMIT is executed, the transaction ends and all savepoints are lost. Attempting to rollback afterward causes an error. Savepoints must be used within the same active transaction.

Question 7c:

Explanation: No, user_viewer cannot access the table. DCL commands such as GRANT and REVOKE are auto-committed and are not affected by ROLLBACK. Even though a rollback is issued, the revoke operation has already taken effect. Therefore, the user permanently loses access.